Solace-AI: AWS Staging IAM Setup

Granting the deployer permissions to stand up the staging environment

Date	2026-08-24	**Account**	253025918446
Region	us-east-1	**Scope**	solace-staging-* resources only

1. Overview

This guide grants a deployer principal exactly the permissions needed to stand up the Solace-AI staging environment (a single EC2 host running the project docker-compose stack, created with CloudFormation) and nothing more. The configured CLI user (MedCodex-Local) currently has only Bedrock, Aurora DSQL, and read-only IAM/quotas access, so it cannot create EC2, CloudFormation, or VPC resources. This document fixes that.

There are two options. Pick one:

- **Option A (recommended):** create a new dedicated IAM user `solace-staging-deployer`. Smallest blast radius, easy to delete after staging, and it does not broaden your everyday inference user.
- **Option B (fastest):** attach the same permissions to the existing `MedCodex-Local` user. Fewer steps, but it widens a shared account user.

Either way, all IAM write actions below must be run by an account administrator (AWS Console, or an admin CLI profile). MedCodex-Local itself cannot perform them.

Permissions this grants (and why)

Permission	Why it is needed
AWSCloudFormationFullAccess	Create / update / delete the solace-staging stack.
AmazonEC2FullAccess	Launch and terminate the staging host, security group, and EBS volume.
AmazonSSMFullAccess	Keyless shell access via Session Manager (no open SSH), and reading the AMI parameter.
Inline: IAM role + instance profile (scoped solace-staging-*)	The EC2 instance needs an SSM-enabled role; the deployer creates it.
Inline: iam:PassRole to ec2 (scoped)	Attach that role to the instance at launch.
Inline: budgets (scoped solace-staging-*)	Create the cost-guardrail alarm.

Tip: the easiest place to run any CLI below is AWS CloudShell (a browser bash terminal in the AWS Console) signed in as an admin. It avoids all local shell and credential setup. Upload solace-staging-deployer-policy.json there first, or paste the JSON via the Console.

2. Option A: create a dedicated IAM user (recommended)

Console breadcrumbs

- Sign in to the AWS Console as an administrator, then open IAM: `https://console.aws.amazon.com/iam/`
- Users -> Create user. User name: `solace-staging-deployer`. Leave 'Provide user access to the Console' UNCHECKED (programmatic access only).
- Set permissions -> Attach policies directly. Search for and check all three: `AWSCloudFormationFullAccess`, `AmazonEC2FullAccess`, `AmazonSSMFullAccess`. Next -> Create user.

- Open the new user -> Permissions -> Add permissions -> Create inline policy -> JSON tab. Paste the policy from Section 4. Next, name it `solace-staging-deployer-iam-budgets`, Create policy.
- Security credentials tab -> Create access key -> 'Application running outside AWS' -> Create. Download the .csv (Access key ID + Secret access key). You will not see the secret again.
- Configure a local CLI profile named `solace-staging` with those keys (below).

CLI breadcrumbs (admin profile)

```
# Set your admin CLI profile name once.
ADMIN=your-admin-profile

aws iam create-user \
  --user-name solace-staging-deployer --profile "$ADMIN"

# Attach the three AWS-managed policies.
for P in AWSCloudFormationFullAccess AmazonEC2FullAccess AmazonSSMFullAccess; do
  aws iam attach-user-policy \
    --user-name solace-staging-deployer \
    --policy-arn arn:aws:iam::aws:policy/$P \
    --profile "$ADMIN"
done

# Attach the scoped inline policy (run from this folder so file:// resolves).
aws iam put-user-policy \
  --user-name solace-staging-deployer \
  --policy-name solace-staging-deployer-iam-budgets \
  --policy-document file://solace-staging-deployer-policy.json \
  --profile "$ADMIN"

# Create an access key for the deployer (capture the output).
aws iam create-access-key \
  --user-name solace-staging-deployer --profile "$ADMIN"
```

Configure the solace-staging profile

```
aws configure set aws_access_key_id <ACCESS_KEY_ID> --profile solace-staging
aws configure set aws_secret_access_key <SECRET_ACCESS_KEY> --profile solace-staging
aws configure set region us-east-1 --profile solace-staging
```

Paste the secret access key only into your own terminal. Do not send it to Claude or into any chat, URL, header, or file that leaves your machine.

3. Option B: use the existing MedCodex-Local user (fastest)

Same three managed policies plus the same inline policy, attached to the existing user. No new access key needed - Claude keeps using the current default / medcodex-local profile.

Console breadcrumbs

- IAM -> Users -> MedCodex-Local -> Add permissions -> Attach policies directly. Check the three managed policies, then Add permissions.
- Add permissions -> Create inline policy -> JSON. Paste Section 4, name it `solace-staging-deployer-iam-budgets`, Create policy.

CLI breadcrumbs (admin profile)

```
ADMIN=your-admin-profile

for P in AWSCloudFormationFullAccess AmazonEC2FullAccess AmazonSSMFullAccess; do
  aws iam attach-user-policy \
    --user-name MedCodex-Local \
    --policy-arn arn:aws:iam::aws:policy/$P \
    --profile "$ADMIN"
done

aws iam put-user-policy \
  --user-name MedCodex-Local \
  --policy-name solace-staging-deployer-iam-budgets \
  --policy-document file://solace-staging-deployer-policy.json \
  --profile "$ADMIN"
```

4. The scoped inline policy (copy-paste)

This is the least-privilege inline policy referenced above. It is also saved in the repo at `infrastructure/aws-staging/solace-staging-deployer-policy.json`. Every action is scoped to solace-staging-* resources in account 253025918446.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SolaceStagingInstanceProfile",
      "Effect": "Allow",
      "Action": [
        "iam:CreateRole",
        "iam:DeleteRole",
        "iam:GetRole",
        "iam:TagRole",
        "iam:PutRolePolicy",
        "iam:DeleteRolePolicy",
        "iam:GetRolePolicy",
        "iam:ListRolePolicies",
        "iam:ListAttachedRolePolicies",
        "iam:CreateInstanceProfile",
        "iam:DeleteInstanceProfile",
        "iam:GetInstanceProfile",
        "iam:AddRoleToInstanceProfile",
        "iam:RemoveRoleFromInstanceProfile",
        "iam:DetachRolePolicy"
      ],
      "Resource": [
        "arn:aws:iam::253025918446:role/solace-staging-*",
        "arn:aws:iam::253025918446:instance-profile/solace-staging-*"
      ]
    },
    {
      "Sid": "SolaceStagingAttachSsmCoreOnly",
      "Effect": "Allow",
      "Action": "iam:AttachRolePolicy",
      "Resource": "arn:aws:iam::253025918446:role/solace-staging-*",
      "Condition": {
        "StringEquals": {
          "iam:PolicyARN": "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
        }
      }
    },
    {
      "Sid": "SolaceStagingPassRoleToEc2",
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::253025918446:role/solace-staging-*",
      "Condition": {
        "StringEquals": {
          "iam:PassedToService": "ec2.amazonaws.com"
        }
      }
    },
    {
      "Sid": "SolaceStagingBudgets",
      "Effect": "Allow",
      "Action": [
        "budgets:ViewBudget",
        "budgets:CreateBudget",
        "budgets:ModifyBudget",
        "budgets:DeleteBudget"
      ],
      "Resource": "arn:aws:budgets::253025918446:budget/solace-staging-*"
    }
  ]
}
```

5. Verify the grant

Run these with the target profile (solace-staging for Option A, or default / medcodex-local for Option B). Success means no AccessDenied. describe-instances previously returned UnauthorizedOperation and should now succeed.

```
aws sts get-caller-identity --profile solace-staging
aws cloudformation list-stacks --profile solace-staging --region us-east-1
aws ec2 describe-vpcs --filters "Name=isDefault,Values=true" \
  --profile solace-staging --region us-east-1
aws ec2 describe-instances --max-items 1 \
  --profile solace-staging --region us-east-1
```

6. What happens next

When the grant is in place, tell Claude: (a) the policies are attached, and (b) which profile to use. Claude will then:

- Re-probe permissions to confirm the grant took effect.
- Draft a `solace-staging` CloudFormation template: default VPC, one m5.xlarge EC2 host, 60 GB gp3 volume, an SSM-enabled instance profile, a security group with no open inbound ports (Session Manager only), and a solace-staging Budgets alarm at $50/month.
- Walk you through the apply and show the estimated cost BEFORE anything billable is created.
- Run docs/STAGING-BRINGUP.md on the host (build images, migrate, health-check), then prove the infra-gated verification bucket (RLS + GDPR erasure on real Postgres, crisis end-to-end on real Kafka, LangGraph checkpointer, outbox two-replica, TLS floors, CI docker gates).
- Tear the stack down when verification is complete.

Data safety: staging uses synthetic data only. No live or real PHI until the HIPAA BAA is signed. The startup guards now also require real POSTGRES_PASSWORD / SECRET_KEY / ENCRYPTION_KEY and POSTGRES_SSL_MODE at least 'require' - Claude sets these in the staging .env.prod during bring-up.

7. Teardown and revoke (after staging)

```
# 1) Delete the staging stack.
aws cloudformation delete-stack --stack-name solace-staging \
  --profile solace-staging --region us-east-1

# 2a) Option A - remove the dedicated deployer entirely.
aws iam delete-access-key --user-name solace-staging-deployer \
  --access-key-id <ACCESS_KEY_ID> --profile "$ADMIN"
for P in AWSCloudFormationFullAccess AmazonEC2FullAccess AmazonSSMFullAccess; do
  aws iam detach-user-policy --user-name solace-staging-deployer \
    --policy-arn arn:aws:iam::aws:policy/$P --profile "$ADMIN"
done
aws iam delete-user-policy --user-name solace-staging-deployer \
  --policy-name solace-staging-deployer-iam-budgets --profile "$ADMIN"
aws iam delete-user --user-name solace-staging-deployer --profile "$ADMIN"

# 2b) Option B - just remove the grants from MedCodex-Local.
for P in AWSCloudFormationFullAccess AmazonEC2FullAccess AmazonSSMFullAccess; do
  aws iam detach-user-policy --user-name MedCodex-Local \
    --policy-arn arn:aws:iam::aws:policy/$P --profile "$ADMIN"
done
aws iam delete-user-policy --user-name MedCodex-Local \
  --policy-name solace-staging-deployer-iam-budgets --profile "$ADMIN"
```

8. Security notes

- The inline policy is scoped to solace-staging-* ARNs only; it cannot touch MedCodex resources.

- Access to the host is via SSM Session Manager - no inbound SSH, no key pairs, no public ports.
- The stack is ephemeral and cost-capped; tear it down after the verification bucket is proven.
- Resource names use hyphens (no underscores or em dashes), per account naming standards.
- Prefer Option A: deleting one dedicated user cleanly revokes everything when staging is done.